



**Master of Computer Application – Game Development
Semester – I**

Sub: Front-End Frameworks and Technologies

Topic: Dynamic To-Do List Application

By

Name: Shaurya Venkta

Reg no.: 1340826015

Faculty Name: VEERA RAGHAV K

Faculty Signature: _____

**Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106**

October 2026

INDEX

FIG NO.	NAME	PG NO.
1	INTRODUCTION	3
2	INPUT CODE	4-12
3	OUTPUT	13
4	CONCLUSION	14

INTRODUCTION

The Dynamic To-Do List Application is a client-side web application developed using HTML5, CSS3, and JavaScript to create and manage daily tasks efficiently. It allows users to add, edit, delete, and mark tasks as completed, while automatically displaying the current number of pending tasks.

The application uses JavaScript arrays, objects, functions, conditional statements, loops, DOM manipulation, event handling, and Local Storage to provide dynamic task management and data persistence. It also includes All, Pending, and Completed filters, along with a clear completed tasks functionality for better user interaction.

Overall, this project demonstrates the practical use of JavaScript-based data management, dynamic content rendering, browser storage, and interactive web development in creating a simple and user-friendly task management system.

.

INPUT CODE

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Dynamic To-Do List Application</title>
    <style>
      body {
        margin: 0;
        background: #111;
        font-family: Arial, sans-serif;
      }
      .wrap {
        width: 90%;
        max-width: 1100px;
        margin: 40px auto;
        background: #f2f2f2;
        border-top: 1px solid #666;
        border-bottom: 1px solid #666;
        padding: 25px 30px 30px;
      }
      h1 {
        font-size: 2.4rem;
        margin: 10px 0 20px;
      }
      h1 span {
        font-size: 1.7rem;
      }
      .problem {
        font-size: 1.1rem;
        line-height: 1.7;
        margin-bottom: 20px;
      }
```

```
}  
.problem b {  
  font-size: 1.3rem;  
}  
ul {  
  padding-left: 25px;  
  line-height: 2;  
  font-size: 1.1rem;  
}  
.tech {  
  font-weight: bold;  
  font-size: 1.1rem;  
  margin: 20px 0;  
}  
.box {  
  margin-top: 30px;  
  background: #fff;  
  border: 1px solid #ddd;  
  border-radius: 8px;  
  padding: 18px;  
}  
.row {  
  display: flex;  
  gap: 10px;  
  margin-bottom: 15px;  
}  
input {  
  flex: 1;  
  padding: 10px;  
  font-size: 1rem;  
}  
button {  
  padding: 10px 16px;  
  background: #111;  
  color: #fff;  
  border: none;  
  cursor: pointer;
```

```
}  
.filters {  
  display: flex;  
  gap: 10px;  
  margin: 10px 0;  
}  
.clearBtn {  
  background: #ddd;  
  color: #111;  
  margin-bottom: 10px;  
}  
.count {  
  margin: 10px 0;  
  font-weight: bold;  
}  
.item {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 10px;  
  background: #f9f9f9;  
  border: 1px solid #eee;  
  padding: 10px;  
  margin-top: 8px;  
}  
.left {  
  display: flex;  
  align-items: center;  
  gap: 8px;  
}  
.done {  
  text-decoration: line-through;  
  color: #777;  
}  
.del {  
  background: #d9534f;  
}
```

```

</style>
</head>
<body>
  <div class="wrap">
    <h1>Dynamic To-Do List Application</h1>
    <div class="problem">
      <b>Problem Statement:</b><br />Develop a dynamic To-Do List
application
      that allows users to create and manage tasks based on their completion
      status.
    </div>
    <div><b>Features</b></div>
    <ul>
      <li>Add tasks</li>
      <li>Edit tasks</li>
      <li>Delete tasks</li>
      <li>Mark completed</li>
      <li>Pending/completed filters</li>
      <li>Task count</li>
      <li>Clear completed tasks</li>
      <li>Local storage</li>
    </ul>
    <div class="tech">Technology: HTML, CSS, JavaScript</div>

    <div class="box">
      <div class="row">
        <input id="task" type="text" placeholder="Add a new task..." />
        <button id="add">Add Task</button>
      </div>

      <div class="filters">
        <button class="filter active" data-filter="all">All</button>
        <button class="filter" data-filter="pending">Pending</button>
        <button class="filter" data-filter="completed">Completed</button>
      </div>

      <button class="clearBtn" id="clearCompleted">Clear Completed</button>

```

```

    <div class="count" id="count">0 tasks left</div>
    <ul id="list"></ul>
  </div>
</div>

<script>
  let tasks = JSON.parse(localStorage.getItem("todoList")) || [];
  let activeFilter = "all";

  const taskInput = document.getElementById("task");
  const addBtn = document.getElementById("add");
  const list = document.getElementById("list");
  const count = document.getElementById("count");
  const filterButtons = document.getElementsByClassName("filter");
  const clearCompletedBtn = document.getElementById("clearCompleted");

  function saveTasks() {
    localStorage.setItem("todoList", JSON.stringify(tasks));
  }

  function filteredTasks() {
    let result = [];

    for (let i = 0; i < tasks.length; i++) {
      if (activeFilter === "all") {
        result.push(tasks[i]);
      } else if (activeFilter === "pending" && tasks[i].done === false) {
        result.push(tasks[i]);
      } else if (activeFilter === "completed" && tasks[i].done === true) {
        result.push(tasks[i]);
      }
    }
  }

  return result;
}

function showTasks() {

```

```
list.innerHTML = "";
let remaining = 0;
let items = filteredTasks();

for (let i = 0; i < tasks.length; i++) {
  if (tasks[i].done === false) {
    remaining = remaining + 1;
  }
}

for (let i = 0; i < tasks.length; i++) {
  let task = tasks[i];

  if (activeFilter === "pending" && task.done === true) {
    continue;
  }

  if (activeFilter === "completed" && task.done === false) {
    continue;
  }

  let row = document.createElement("li");
  row.className = "item";

  let leftPart = document.createElement("div");
  leftPart.className = "left";

  let check = document.createElement("input");
  check.type = "checkbox";
  check.checked = task.done;
  check.onclick = function () {
    tasks[i].done = !tasks[i].done;
    saveTasks();
    showTasks();
  };

  let text = document.createElement("span");
```

```

text.innerText = task.text;
if (task.done) {
  text.className = "done";
}

leftPart.appendChild(check);
leftPart.appendChild(text);

let rightPart = document.createElement("div");

let editBtn = document.createElement("button");
editBtn.innerText = "Edit";
editBtn.onclick = function () {
  let newText = prompt("Edit task", task.text);
  if (newText !== null && newText.trim() !== "") {
    tasks[i].text = newText.trim();
    saveTasks();
    showTasks();
  }
};

let deleteBtn = document.createElement("button");
deleteBtn.innerText = "Delete";
deleteBtn.className = "del";
deleteBtn.onclick = function () {
  tasks.splice(i, 1);
  saveTasks();
  showTasks();
};

rightPart.appendChild(editBtn);
rightPart.appendChild(deleteBtn);

row.appendChild(leftPart);
row.appendChild(rightPart);
list.appendChild(row);
}

```

```

    count.innerText = remaining + " tasks left";
}

for (let i = 0; i < filterButtons.length; i++) {
    filterButtons[i].onclick = function () {
        activeFilter = filterButtons[i].getAttribute("data-filter");

        for (let j = 0; j < filterButtons.length; j++) {
            filterButtons[j].classList.remove("active");
        }

        filterButtons[i].classList.add("active");
        showTasks();
    };
}

clearCompletedBtn.onclick = function () {
    let newTasks = [];

    for (let i = 0; i < tasks.length; i++) {
        if (tasks[i].done === false) {
            newTasks.push(tasks[i]);
        }
    }

    tasks = newTasks;
    saveTasks();
    showTasks();
};

addBtn.onclick = function () {
    let text = taskInput.value.trim();

    if (text !== "") {
        tasks.push({ text: text, done: false });
        taskInput.value = "";
    }
}

```

```
        saveTasks();
        showTasks();
    }
};

taskInput.onkeydown = function (event) {
    if (event.key === "Enter") {
        addBtn.click();
    }
};

showTasks();
</script>
</body>
</html>
```

OUTPUT

Dynamic To-Do List Application

Problem Statement:

Develop a dynamic To-Do List application that allows users to create and manage tasks based on their completion status.

Features

- Add tasks
- Edit tasks
- Delete tasks
- Mark completed
- Pending/completed filters
- Task count
- Clear completed tasks
- Local storage

Technology: HTML, CSS, JavaScript

Add Task

All Pending Completed

Clear Completed

0 tasks left

CONCLUSION

The Dynamic To-Do List Application successfully demonstrates how HTML, CSS, and JavaScript can be combined to create a simple, interactive, and user-friendly task management system. By using JavaScript for task creation, editing, deletion, completion tracking, and filtering, the application provides an efficient way to organize and manage daily tasks without requiring server-side processing.

The system effectively manages tasks using JavaScript arrays and objects, while features such as Pending/Completed filters, task count, and Clear Completed improve usability and interaction. The use of Local Storage also allows task data to be saved in the browser and retained even after the page is refreshed.

Overall, this project demonstrates fundamental web development concepts such as DOM manipulation, event handling, functions, arrays, objects, loops, conditional statements, dynamic content rendering, and client-side data storage in creating a practical task management application.